Calcul Scientifique

SMP5

Rachid Moustabchir
Departement de Physique

2025/11/14

Plan du cours

- Chapitre I: Introduction à l'Algorithmique Numérique avec Python et Visualisation des Données
- Chapitre II: Équations Différentielles Ordinaires (ODE)
- Chapitre III: Stabilité des systèmes dynamiques
- **Chapitre IV: Résolution numérique des E.D.P.**

Plan

- Exemples d'E.D.P.
  - Equation de Laplace/Poisson
  - Equation de la chaleur
  - Equation des ondes
- Méthodes de résolution numérique d'EDP
- Méthode des différences finies 1D
  - EDP stationnaire 1D + Dirichlet
  - EDP stationnaire + CL mixtes
- Problème modèle évolutif
  - Schéma explicite
  - Schéma implicite

Exemples d'EDP : Equation de Laplace et équation de Poisson



Pierre-Simon Laplace
1749-1827,
mathématicien, astronome, physicien et homme politique français



Siméon Denis Poisson
1781-1840,
mathématicien, géomètre et physicien français

- Soit l'equation de Poisson: $-\Delta u = f$ ***dans*** $\Omega \subset \mathbb{R}^n$
- où u et f sont des functions de $\mathbb{R}^n$ dans $\mathbb{R}$ et Δ est l'opérateur laplacien : $\Delta u = \sum_{i=1}^{n} \frac{\partial^2 u}{\partial x_i^2} = \mathrm{div}(\vec{\nabla} \boldsymbol{u})$.
- C'est l'équation modèle pour tous les problèmes elliptiques.
  - $f \equiv 0$: équation de **Laplace**.
  - $f \not\equiv 0$: équation de **Poisson**.

Exemples d'EDP : Equation de Laplace et équation de Poisson

Exemples physiques :

- En électrostatique où elle s'écrit : $\Delta V = -\frac{\rho}{\epsilon_0}$,
  - ρ désigne la densité volumique de charge électrique, ϵ_0 la permittivité électrique du vide, et V le potentiel électrique relié au champ élec. $\vec{E}$ par la relation : $\vec{E} = -\vec{\nabla}V$
  - Elle traduit le fait que $\vec{E}$ doit vérifier l'équation de Gauss: $div(\vec{E}) = -\frac{\rho}{\epsilon_0}$
- En thermique (en régime stationnaire) où elle s'écrit : $\Delta T = -\frac{f}{\lambda}$,
  - T : la température en chaque point d'un solide homogène de conductivité thermique λ et f modélise la quantité d'énergie libérée par unité de temps et de volume au sein du solide (par effet Joule par exemple).
  - Elle traduit la conservation locale de l'énergie : $div(\vec{q}) = f$,où $\vec{q} = -\lambda\vec{\nabla}T$est le flux de chaleur donné par la loi de Fourier.
- En dynamique des fluides parfaits incompressibles où elle s'écrit : $\Delta\phi = 0$,
  - ϕ désigne le **potentiel des vitesses** reliée à la vitesse $\vec{V}$ du fluide par : $\vec{V} = \vec{\nabla}\phi$
  - traduit le fait que le champ de vitesse doit vérifier l'équation de conservation du volume (cas d'un fluide incompressible) : $div(\vec{V}) = 0$

Exemples d'EDP : Equation de Laplace et équation de Poisson

Conditions aux limites

- L'équation de Poisson est une EDP linéaire du second ordre.
- Il est évident que **sa solution n'est pas unique**.
  - Si u est solution, toute fonction de la forme $u + v$ avec $v = \mathbf{a}^\top \mathbf{x} + b = \sum_{i=1}^{n} a_i x_i + b$ (fonction affine de $x_i \to \Delta v = \sum_{i=1}^{n} \frac{\partial^2 v}{\partial x_i^2} = 0$) est également solution.
- Pour que le modèle possède **une unique solution**, il faut **imposer une condition aux limites sur u**.
- Si u est définie sur tout l'espace (c'est-à-dire sur $\mathbb{R}^n$ tout entier) on impose le comportement de u à l'infini. Par exemple, en électrostatique, on impose que le potentiel tende vers 0 à l'infini.
- Si on veut résoudre numériquement l'équation, la fonction u n'est pas définie sur tout l'espace mais seulement sur un sous domaine borné Ω de $\mathbb{R}^n$.
  - On impose alors à u de vérifier une certaine condition sur la frontière $\partial\Omega$ de Ω.
  - Le choix de cette condition fait partie de la modélisation du problème au même titre que l'EDP.

Exemples d'EDP : Conditions aux limites

Condition de Dirichlet (valeur imposée de la fonction)

$$u = g_D \quad \textbf{sur} \quad \Gamma_D \subset \partial\Omega$$

- On fixe directement la valeur de la fonction u sur une partie du bord Γ_D.

Condition de Neumann (valeur imposée de la dérivée normale)

$$\frac{\partial u}{\partial n} = \vec{\nabla} u(x).\ \vec{n}(x) = g_N \quad \textbf{sur} \quad \Gamma_N \subset \partial\Omega$$

- où $\frac{\partial u}{\partial n}$ est la dérivée de u selon la normale au bord.
- On impose le flux sortant ou entrant à travers la frontière.

Condition de Robin (combinaison de valeur et flux)

$$\frac{\partial u}{\partial n} + \alpha u = g_R \quad \textbf{sur} \quad \Gamma_R \subset \partial\Omega \text{ où } \alpha \text{ est un coefficient.}$$

- Mélange entre Dirichlet et Neumann : la valeur de u et son flux sont liés.

- α, g_D, g_N, g_R sont des données (expér. ou issues de la modélisation du prob.).
- La condition de Robin est la plus générale. Dans le cas $\alpha = 0$ elle redonne la condition de Neumann et dans le cas $\alpha = +\infty$ elle redonne la condition de Dirichlet.

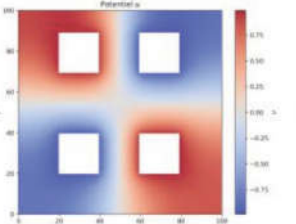
Exemples d'EDP : Conditions aux limites

Applications 1: Problème de condensateur en 2D

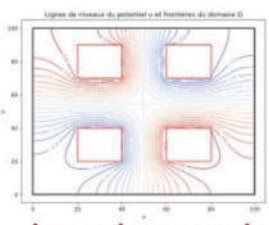
- Trouver le le potentiel $u: \Omega \to \mathbb{R}$ tel que:

$$
\begin{aligned}
&-\Delta u = 0 \quad \text{dans } \Omega \subset \mathbb{R}^2,\\
&u = 0 \quad \text{sur } \Gamma_{10},\\
&u = -1 \quad \text{sur } \Gamma_2 \cup \Gamma_3,\\
&u = 1 \quad \text{sur } \Gamma_1 \cup \Gamma_4.\\
&\frac{\partial u}{\partial n} = 0 \quad \text{sur } \Gamma_{10}.
\end{aligned}
$$

Solution numérique utilisant un schéma aux différences finies d'ordre 2

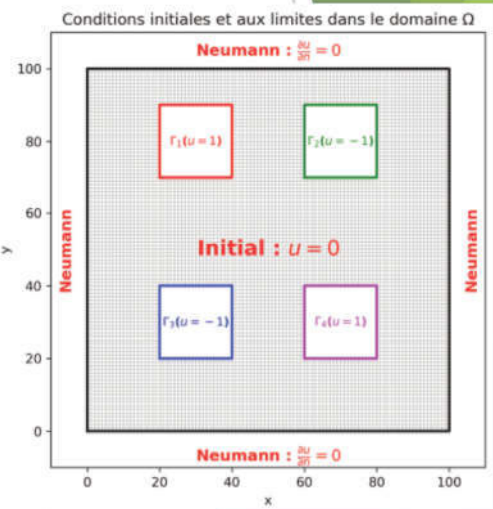


Potentiel u



Lignes de niveaux du potentiel u

frontières du domaine Ω



Exemples d'EDP : Conditions aux limites

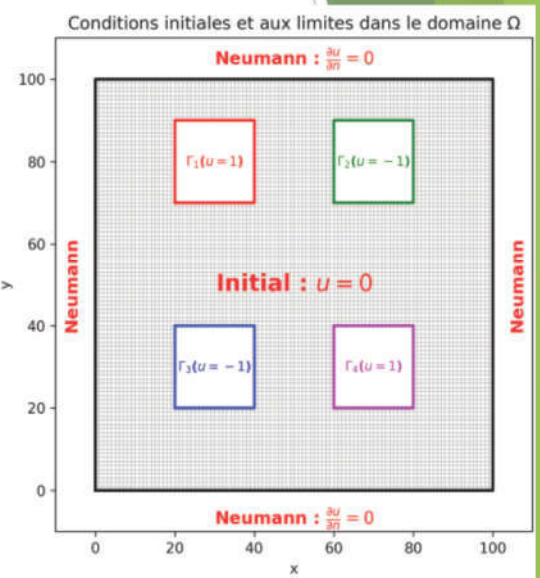
frontières du domaine Ω

Applications 2: champ de vitesses

- Trouver le potentiel $u: \Omega \to \mathbb{R}$ tel que:

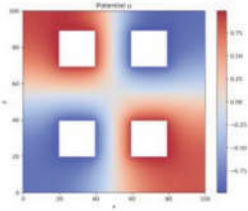
$$
\begin{aligned}
-\Delta u &= 0 \quad \text{dans } \Omega \subset \mathbb{R}^2, \\
u &= 0 \quad \text{sur } \Gamma_{10}, \\
u &= -1 \quad \text{sur } \Gamma_2 \cup \Gamma_3, \\
u &= 1 \quad \text{sur } \Gamma_1 \cup \Gamma_4, \\
\frac{\partial u}{\partial n} &= 0 \quad \text{sur } \Gamma_{10}.
\end{aligned}
$$

- Le champ de vitesses associé est donné par: $\vec{V} = \vec{\nabla} u$

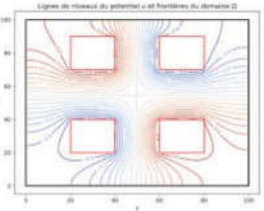


Exemples d'EDP : Conditions aux limites

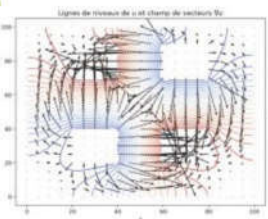
Applications 2: champ de vitesses $\vec{V} = \vec{\nabla} u$



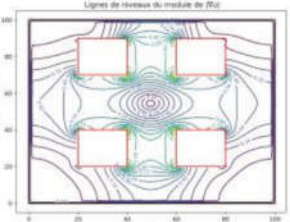
Potentiel u



Lignes de niveaux du potentiel u



Lignes de niveaux du potentiel u et champ de vecteurs $\vec{V}$



Lignes de niveaux de V

Solution numérique utilisant un schéma aux différences finies d'ordre 2

Exemples d'EDP : Conditions aux limites

Applications 3:Problème mal posé

- Trouver $u: \Omega \to \mathbb{R}$ tel que
  $$-\Delta u = f \quad \text{dans } \Omega \subset \mathbb{R}^n,$$
  $$\frac{\partial u}{\partial n} = g \quad \text{sur } \partial\Omega.$$
- **Problème en dimension** n où Ω est un domaine borné de $\mathbb{R}^n$.
- Ce problème est **mal posé** car il n'y a pas unicité de la solution.
  - Si u est solution du problème, alors $u(x) + C$ (avec C constante) est aussi solution.
  - $u(x) + C$ a **exactement la même dérivée normale** : $\frac{\partial (u+C)}{\partial n} = \frac{\partial u}{\partial n} + 0 = g$
  - $\frac{\partial u}{\partial n}$ fixe uniquement la pente de u au bord, **pas son niveau absolu** (valeur u elle-même).
- Pour obtenir l'unicité, il faudrait imposer, sur une partie non vide du bord, une condition de Dirichlet ou une condition de Robin
  - **Dirichlet :** on impose $u = h$ sur $\partial\Omega$ → valeur de u fixée sur le bord → unicité garantie.
  - **Neumann :** on impose $\frac{\partial u}{\partial n} = g$ → seulement la pente fixée sur le bord → unicité perdue sans condition supplémentaire.

Exemples d'EDP : Equation de la chaleur

- L' équation de la chaleur

$$\frac{\partial u}{\partial t}(t,x) - D\Delta u(t,x) = \frac{f(t,x)}{\rho c}, \quad \forall x \in \Omega, \quad \forall t \in [0,T]$$

  - $\Omega \subset \mathbb{R}^d$ de frontière $\partial\Omega$ et T est le temps final
  - D, coefficient de diffusivité thermique (en m^2/s),
  - f, production volumique de chaleur (en W/m^3),
  - ρ, masse volumique du matériau (en kg/m^3),
  - c, chaleur spécifique massique du matériau (en $J/kg/K$),
  - $u(t,x)$, température au point x et à l'instant t.

- **Problème bien posé ?**

Exemples d'EDP : Equation de la chaleur

Conditions de Cauchy (Condition initiale):

$$u(0,x) = u_0(x), \quad \forall x \in \Omega$$

- où u_0 est l'état initial.

Conditions aux limites sur $\partial\Omega$:

Dirichlet (valeur imposée)

- $u(t,x) = g_D(t,x), \quad \forall x \in \Gamma_D \subset \partial\Omega, \ \forall t \in [0,T]$

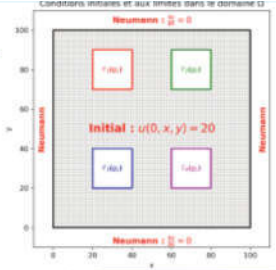
Neumann (flux imposé)

- $D\frac{\partial u}{\partial n}(t,x) = g_N(t,x), \quad \forall x \in \Gamma_N \subset \partial\Omega, \ \forall t \in [0,T]$

Robin (mélange valeur + flux)

- $D\frac{\partial u}{\partial n}(t,x) + \alpha u(t,x) = g_R(t,x), \quad \forall x \in \Gamma_R \subset \partial\Omega, \ \forall t \in [0,T]$

Exemples d'EDP : Equation de la chaleur



frontières du domaine Ω

Applications 4: Problème de chaleur en 2D

- On cherche $u: [0,T] \times \Omega \to \mathbb{R}$ tel que :

$$\frac{\partial u}{\partial t} - D\,\Delta u = 0 \quad \text{dans} \quad [0,T] \times \Omega, \text{ avec } \Omega \subset \mathbb{R}^2$$

$$u(0,x,y) = 20\ \forall (x,y) \in \Omega,\ \frac{\partial u}{\partial n} = 0 \text{ sur } \Gamma_{10},\ u = g_1 \text{ sur } \Gamma_2 \cup \Gamma_3,\ u = g_2 \text{ sur } \Gamma_1 \cup \Gamma_4.$$

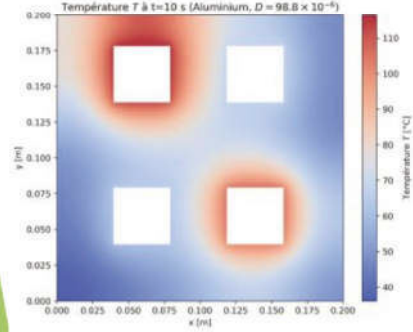
- Domaine Ω : carré de côté $20\,\text{cm}$.
- Coefficient de diffusion thermique D de l'Aluminium : $D = 98.8 \times 10^{-6}$
- Conditions aux limites :
  - Pour $x \in \Gamma_2 \cup \Gamma_3$: $g_1(t,x,y) = \begin{cases} 20 + 40t + \frac{10y}{L} & \text{si } t \leq 1, \\ 60 + \frac{10y}{L} & \text{sinon.} \end{cases}$
  - Pour $x \in \Gamma_1 \cup \Gamma_4$: $g_2(t,x,y) = \begin{cases} 20 + 80t + \frac{20y}{L} & \text{si } t \leq 1, \\ 100 + \frac{20y}{L} & \text{sinon.} \end{cases}$
- La température imposée sur les zones Γ_i dépend non seulement du temps mais aussi de la coordonnée verticale y selon une loi linéaire en y

Exemples d'EDP : Equation de la chaleur

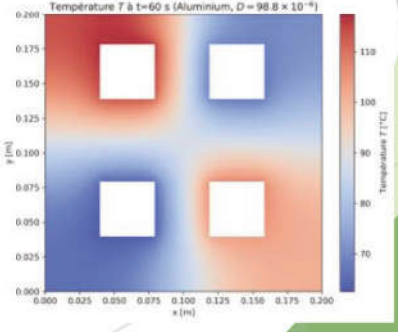
Applications 4: Problème de chaleur en 2D

- Deux solutions numériques de ce problème, obtenues à l'aide d'un schéma aux différences finies d'ordre 2,
- Les graphiques montrent la répartition de la température $u(t,x)$:

$t = 10\,s$



$t = 60\,s$



Exemples d'EDP : Equation des ondes

$$\frac{\partial^2 u}{\partial t^2}(t,x) - c^2 \Delta u(t,x) = 0, \quad \forall x \in \Omega, \quad \forall t \in [0,T]$$

- $\Omega \subset \mathbb{R}^d$ de frontière $\partial\Omega$ et T est le temps final
- $c > 0$ *vitesse de propagation de l'onde.*
- **Problème bien posé ?**

conditions initiales

- $u(0,\mathbf{x}) = u_0(\mathbf{x}), \quad \forall \mathbf{x} \in \Omega$ [position initiale]
- $\frac{\partial u}{\partial t}(0,\mathbf{x}) = v_0(\mathbf{x}), \quad \forall \mathbf{x} \in \Omega$ [vitesse initiale]

conditions aux limites sur $\partial\Omega$

- **Dirichlet** : $\forall \mathbf{x} \in \Gamma_D \subset \partial\Omega, \quad \forall t \in [0,T], \quad u(t,\mathbf{x}) = g_D(t,\mathbf{x})$
- **Neumann** : $\forall \mathbf{x} \in \Gamma_N \subset \partial\Omega, \quad \forall t \in [0,T], \quad c^2 \frac{\partial u}{\partial n}(t,\mathbf{x}) = g_N(t,\mathbf{x})$
- **Robin** : $\forall \mathbf{x} \in \Gamma_R \subset \partial\Omega, \quad \forall t \in [0,T], \quad c^2 \frac{\partial u}{\partial n}(t,\mathbf{x}) + \alpha u(t,\mathbf{x}) = g_R(t,\mathbf{x})$

Définition d'une EDP

Équation aux dérivées partielles (EDP)

- Fait intervenir **plusieurs variables indépendantes** (temps, espace, ...)
- Contient des **dérivées partielles** de la fonction recherchée.
- Forme générale :

$$F\left(x_i, u, \frac{\partial u}{\partial x_i}, \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots\right) = 0$$

Ordre d'une EDP

- **Ordre = ordre le plus élevé** des dérivées présentes.
- Exemple :
  - Convection (ordre 1):

$$\frac{\partial C}{\partial t} + \alpha \frac{\partial C}{\partial x} = 0$$

  - Convection-diffusion (ordre 2):

$$\frac{\partial C}{\partial t} + \alpha \frac{\partial C}{\partial x} - \beta \frac{\partial^2 C}{\partial x^2} = 0$$

Définition d'une EDP

Linéarité

- **Linéaire** : F est linéaire en u et ses dérivées.
- **Quasi-linéaire** :Linéaire uniquement par rapport aux dérivées d'ordre le plus élevé.
- **Non-linéaire** : Coefficients ou termes non-linéaires en u ou ses dérivées.

Exemple d'EDP quasi-linéaire

$$a\frac{\partial^2 u}{\partial x^2} + b\frac{\partial^2 u}{\partial x \partial y} + c\frac{\partial^2 u}{\partial y^2} = d$$

- Quasi-linéaire si a, b, c, d dépendent de $x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}$.

Méthodes de résolution numérique d'EDP

Méthodes déterministes

- Elles consistent à discrétiser l'équation (et éventuellement le domaine) pour obtenir un système d'équations algébriques résoluble par ordinateur.
  - ***Méthode des différences finies (FDM)***
    - **Principe** : Approximer les dérivées par des différences (avant, arrière, centrées).
    - **Avantages** : Simple à implémenter sur des maillages réguliers.
    - **Inconvénients** : Peu flexible pour des géométries complexes.
  - ***Méthode des éléments finis (FEM)***
    - **Principe** : Approximer la solution par des fonctions de forme définies sur des éléments (triangles, quadrilatères, tétraèdres, etc.).
    - **Avantages** : Très adaptée aux géométries complexes et conditions aux limites variées.
    - **Inconvénients** : Mise en œuvre plus complexe que FDM.
  - ***Méthode des volumes finis (FVM)***
    - **Principe** : Intégrer l'EDP sur des volumes de contrôle et appliquer le théorème de Gauss pour transformer les intégrales de volume en intégrales de surface.
    - **Avantages** : Conservation stricte des flux — très utilisée en mécanique des fluides.
    - **Inconvénients** : Plus complexe à implémenter, mais plus robuste pour des lois de conservation.

Méthodes de résolution numérique d'EDP

Méthodes probabilistes

- Elles utilisent des techniques stochastiques pour estimer la solution.
  - ***Méthode de Monte Carlo***
    - **Principe** : Simuler de nombreux scénarios aléatoires (trajectoires, particules) et utiliser les statistiques pour approximer la solution.
    - **Avantages** : Indépendante de la dimension du problème, efficace pour problèmes à grande dimension.
    - **Inconvénients** : Convergence lente, nécessite beaucoup de simulations.

Opérateurs aux différences finies: Dimension 1

- Soit $\varphi: \mathbb{R} \to \mathbb{R}$ une fonction suffisamment régulière.
- On considère un pas $h > 0$ et un point $x \in \mathbb{R}$.
- On définit les opérateurs aux différences finies suivants :
  - **Opérateur aux différences finies progressif (ou décentré avancé)**

$$(D_h^+ \varphi)(x) = \frac{\varphi(x+h) - \varphi(x)}{h}$$

  - **Opérateur aux différences finies rétrograde (ou décentré retardé)**

$$(D_h^- \varphi)(x) = \frac{\varphi(x) - \varphi(x-h)}{h}$$

  - **Opérateur aux différences finies centré**

$$(D_h^0 \varphi)(x) = \frac{\varphi(x+h) - \varphi(x-h)}{2h}$$

Opérateurs aux différences finies: Dimension 1

Approximation d'ordre 2 de la dérivée seconde

- Soit $\varphi: [a,b] \to \mathbb{R}$ et $\varphi^{(k)}$ continue pour $k = 0, \dots, 4$.
- On définit l'opérateur de différence finie centrée d'ordre 2, pour tout $x \in [a,b]$ et $h > 0$ tels que $x \pm h \in [a,b]$, par :

$$(D_h^2\varphi)(x) \overset{\text{def}}{=} \frac{\varphi(x+h) - 2\varphi(x) + \varphi(x-h)}{h^2}$$

- L'opérateur D_h^2 appliqué à φ fournit une **approximation consistante d'ordre 2** de la dérivée seconde $\frac{d^2\varphi}{dx^2}$.
- En outre, on a l'équivalence : $D_h^2\varphi = (D_h^0)^2\varphi = D_h^+(D_h^-\varphi) = D_h^-(D_h^+\varphi)$
- La dérivée seconde peut être obtenue comme **composition de deux opérateurs centrés** $(D_h^0)^2$.
- Elle peut aussi être obtenue par **différence avant suivie de différence arrière** $D_h^+(D_h^-)$.
- Ou encore par **différence arrière suivie de différence avant** $D_h^-(D_h^+)$.

Opérateurs aux différences finies: Dimension n≥1

- Soit $\varphi: U \subset \mathbb{R}^n \to \mathbb{R}$ une fonction suffisamment régulière.
- On note $e_i \in \mathbb{R}^n$ le i-ème vecteur de la base canonique ($i \in \{1, \dots, n\}$).
- Pour $x = (x_1, \dots, x_n) \in U$ et $h > 0$, on définit :
  - **Différence finie progressive selon** x_i

$$(D^+_{h,i}\varphi)(x) = \frac{\varphi(x + he_i) - \varphi(x)}{h}$$

  - **Différence finie rétrograde selon** x_i

$$(D^-_{h,i}\varphi)(x) = \frac{\varphi(x) - \varphi(x - he_i)}{h}$$

  - **Différence finie centrée selon** x_i

$$(D^0_{h,i}\varphi)(x) = \frac{\varphi(x + he_i) - \varphi(x - he_i)}{2h}$$

EDP stationnaire avec conditions aux limites de Dirichlet

Problème modèle

- On considère l'EDP stationnaire 1D suivante :

$$-u''(x) + cu(x) = f(x), \quad x \in [a, b]$$

avec conditions aux limites de Dirichlet : $u(a) = \alpha, \quad u(b) = \beta$

- où $a < b$, $c > 0$, $\alpha, \beta \in \mathbb{R}$ et $f: [a, b] \to \mathbb{R}$ est donnée.
- Il s'agit d'un problème bien posé.
- En général, on ne peut pas résoudre analytiquement ; on utilise donc un **schéma numérique**.

Discrétisation

- On discrétise $[a, b]$ en $N + 1$ points : $x_i = a + ih, \ \ i = 0, \dots, N$ avec pas $\Delta x = h = \frac{b-a}{N}$
- On note $u_i \approx u(x_i)$.

Approximation de la dérivée seconde

$$\varphi''(x) = \frac{\varphi(x+h) - 2\varphi(x) + \varphi(x-h)}{h^2} + O(h^2)$$

$$\Rightarrow u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + O(h^2)$$

EDP stationnaire avec conditions aux limites de Dirichlet

- En remplaçant u'' dans l'EDP :

$$-\frac{u_{i+1}-2u_i+u_{i-1}}{h^2} + cu_i = f(x_i) \text{ pour } i = 1, \dots, N-1 \ (*)$$
$$\Rightarrow -u_{i+1} + 2u_i - u_{i-1} + ch^2 u_i = h^2 f(x_i)$$

- On pose: $\mu = 2 + ch^2$

$\Rightarrow -u_{i-1} + \mu u_i - u_{i+1} = h^2 f(x_i)$ avec conditions : $u_0 = \alpha,\ u_N = \beta$

- Schéma aux différences finies.
- Cela conduit à un **système linéaire tridiagonal**.

$$\Rightarrow \begin{cases} \text{Pour } i = 0: & u_0 = \alpha \Rightarrow h^2 u_0 = h^2 \alpha \\ \text{Pour } i = 1: & -u_2 + \mu u_1 - u_0 = h^2 f(x_1) \\ \text{Pour } i = 2: & -u_3 + \mu u_2 - u_1 = h^2 f(x_2) \\ \text{Pour } i = 3: & -u_4 + \mu u_3 - u_2 = h^2 f(x_3) \\ \text{Pour } i = N-2: & -u_{N-1} + \mu u_{N-2} - u_{N-3} = h^2 f(x_{N-2}) \\ \text{Pour } i = N-1: & -u_N + \mu u_{N-1} - u_{N-2} = h^2 f(x_{N-1}) \\ \text{Pour } i = N: & h^2 u_N = h^2 \beta \end{cases}$$

EDP stationnaire avec conditions aux limites de Dirichlet

Mise en système matriciel

- Si on assemble toutes les équations on obtient :

$$\underbrace{\begin{bmatrix} h^2 & 0 & 0 & \cdots & 0 & 0 \\ -1 & \mu & -1 & \cdots & 0 & 0 \\ 0 & -1 & \mu & \ddots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \ddots & \mu & -1 \\ 0 & 0 & 0 & \cdots & 0 & h^2 \end{bmatrix}}_{\bar{\bar{A}}} \underbrace{\begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-1} \\ u_N \end{bmatrix}}_{\bar{U}} = \underbrace{\begin{bmatrix} h^2\alpha \\ h^2 f(x_1) \\ h^2 f(x_2) \\ \vdots \\ h^2 f(x_{N-1}) \\ h^2\beta \end{bmatrix}}_{\bar{B}}$$

  - Système linéaire de N +1 équations à N+1 inconnues.
  - La matrice A est **tridiagonale** (Chaque équation ne couple que trois inconnues)
  - Seule la 1ère et la dernière ligne diffèrent à cause des conditions de Dirichlet.
- Le schéma (*)+ conditions aux limites est **consistant d'ordre 2** avec l'EDP, et on a:

$$\max_{i\in[0,\dots,N]} |u(x_i) - u_i| = O(h^2)$$

EDP stationnaire avec conditions aux limites de Dirichlet

Exercice

- Q. 1 Écrire la fonction AssembleMat1D retournant la matrice $M \in \mathcal{M}_d(\mathbb{R})$ définie par

$$M = \begin{pmatrix} \gamma & 0 & 0 & \cdots & 0 & 0 \\ \beta & \alpha & \beta & \ddots & \vdots & \vdots \\ 0 & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \beta & \alpha & \beta \\ 0 & \cdots & \cdots & 0 & 0 & \gamma \end{pmatrix} \quad \text{où } \alpha, \beta, \gamma \in \mathbb{R} \text{ sont donnés.}$$

On souhaite résoudre par un schéma aux différences finies l'EDP suivante

$$\begin{aligned} -u'' + cu &= f \quad \text{dans }]a, b[, \\ u(a) &= \alpha, \\ u(b) &= \beta. \end{aligned}$$

- Q. 2 En prenant le jeu de données $a = 0, b = 2\pi, c = 1, \alpha = 1, \beta = -1$ et $f : x \mapsto \cos(x^2)$, écrire une fonction permettant de résoudre l'EDP précédente. On pourra utiliser la fonction $X \leftarrow \text{Solve}(\mathbb{A}, B)$ retournant la solution du système linéaire $\mathbb{A}X = B$.
- Q. 3 En choisissant judicieusement un jeu de données écrire un programme permettant de vérifier l'ordre du schéma utilisé à l'aide de la formule $\max_{i \in [\![0, \ldots N]\!]} |u(x_i) - u_i| = O(h^2)$

EDP stationnaire avec conditions aux limites de Dirichlet

Correction

- R. 1 la fonction AssembleMat1D retournant la matrice $M \in \mathcal{M}_d(\mathbb{R})$

Algorithme Fonction AssembleMat1D

Données : d : dimension de la matrice,
α, β, γ : trois réels
Résultat : M : matrice d×d
1: **Fonction** M←AssembleMat1D(d,α,β,γ)
2: $M \leftarrow 0_(d,d)$
3: $M(1,1) \leftarrow \gamma, M(d,d) \leftarrow \gamma$
4: **Pour** i=2 à d-1 faire
5: $M(i,i) \leftarrow \alpha, M(i,i-1) \leftarrow \beta, M(i,i+1) \leftarrow \beta$
6: **Fin**
7: **Fin**

K.tocsr() sert à convertir la matrice tridiagonale construite en format LIL (modification facile) vers le format CSR (Compressed Sparse Row), qui est :

- Beaucoup plus rapide pour les opérations de produit matrice-vecteur
- Compatible avec les solveurs linéaires
- Plus compact en mémoire

Listing: (Python) Fonction AssembleMatID

```
import numpy as np
from scipy.sparse import lil_matrix
# ----------------------------------------------------
# Assemble la matrice tridiagonale (schéma DF 1D)
# ----------------------------------------------------
def AssembleMatID(d, alpha, beta, gamma):
# Matrice creuse au format LIL (modification efficace)
    K = lil_matrix((d, d))

    # Bord gauche et bord droit
    K[0, 0] = gamma
    K[-1, -1] = gamma

    # Boucle sur les points intérieurs
    for i in range(1, d-1):
        K[i, i]   = alpha     # Diagonale
        K[i, i-1] = beta      # Sous-diagonale
        K[i, i+1] = beta      # Sur-diagonale

    return K.tocsr()
```

- **LIL (lil_matrix) → construire la matrice (ajout d'éléments).**
- **CSR (tocsr()) → calculer (A @ x, solveurs, etc.).**

EDP stationnaire avec conditions aux limites de Dirichlet

Correction

- R. 2 la fonction SolveEDP1 permettant de résoudre l'EDP

Algorithme Fonction SolveEDP1 : résolution de l'EDP par le schéma aux différences finies

Données : a, b : $a < b$,
c : $c > 0$,
α, β : deux réels,
f : fonction de [a,b] à valeurs réelles,
N : nombre de discrétisation
Résultat : x : vecteur de $\mathbb{R}^{n+1}$
U : vecteur de $\mathbb{R}^{n+1}$

1: Fonction $[x, U] \leftarrow SolveEDP1(a, b, c, \alpha, \beta, f, N)$
2: $x \leftarrow DisReg(a, b, N)$
3: $h \leftarrow (b - a)/N$
4: $A \leftarrow AssembleMat1D(N + 1, 2 + ch^2, -1, h^2)$
5: $B(1) \leftarrow h^2\alpha,\ B(N + 1) \leftarrow h^2\beta$
6: `Pour i=2 à N faire`
7: $B(i) \leftarrow h^2 f(x(i))$
8: Fin
9: $U \leftarrow Solve(A, B)$
10: Fin

- $spsolve()$ résout le système $AU = B$ pour U

Listing: (Python) Fonction SolveEDP1

```
from scipy.sparse.linalg import spsolve
# ------------------------------------------------
# Solveur EDP 1D -u'' + c u = f avec Dirichlet
# ------------------------------------------------
def SolveEDP1(a, b, c, alpha, beta, f, N):
# Pas spatial et grille
   h = (b - a) / N
   x = np.linspace(a, b, N + 1)

   # Assemblage de la matrice
   A = AssembleMatID(N + 1, 2 + c * h**2, -1, h**2)

   # Second membre
   B = np.zeros(N + 1)
   B[0] = alpha
   B[-1] = beta
   for i in range(1, N):
      B[i] = f(x[i])
   B = h**2 * B

   # Résolution du système linéaire
   U = spsolve(A, B)

   return x, U
```

EDP stationnaire avec conditions aux limites de Dirichlet

Correction

- R. 3 Nous allons calculer pour différentes valeurs de N $(h = (b-a)/N)$ l'erreur $E(h) = \max_{i\in[0,N]}|u(x_i) - u_i|$ commise.
- Il nous faut donc choisir les données pour avoir une solution analytique.
- on choisit une fonction u suffisament régulière, par exemple $u(x) = \sin(x^2)$, puis on détermine la fonction f par $f(x) = -u''(x) + cu(x)$, avec $c = 1$.

$$\Rightarrow f(x) = 4x^2\sin(x^2) - 2\cos(x^2) + \sin(x^2).$$

  - Les conditions aux limites sont alors $u(a) = \sin(a^2)$ et $u(b) = \sin(b^2)$.
  - Avec $c = 1$, l'EDP admettant une unique solution celle-ci est nécessairement la fonction $\sin(x^2)$.
- Comme pour les EDO, la figure permet grâce à une représentation en échelle logarithmique de retrouver l'ordre $p = 2$ du schéma.

$$E(h) = \mathcal{O}(h^p) \approx Ch^p$$
$$\Rightarrow \mathbf{log}\boldsymbol{E(h)} \approx \log C + \boldsymbol{p}\boldsymbol{log}\boldsymbol{h}$$

EDP stationnaire avec conditions aux limites de Dirichlet

Listing: (Python) Example Extrait du script complet

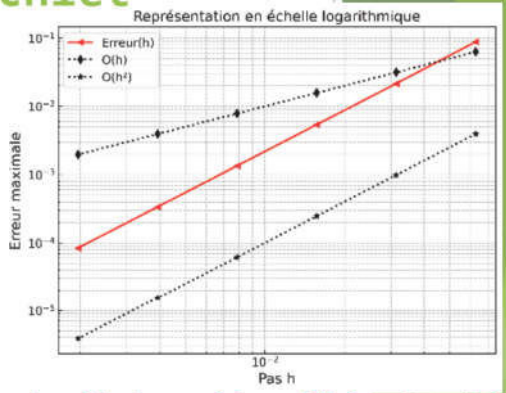
```
def test_convergence_edp():
    # Données du problème
    u_exact = lambda x: np.sin(x**2)  # Solution exacte
    c = 1
    f = lambda x: 4*x**2*np.sin(x**2) - 2*np.cos(x**2) +
c*np.sin(x**2)
    a, b = 0, 2*np.pi

    # Différentes discrétisations
    LM = [100, 200, 400, 800, 1600, 3200]
    H = []
    E = []

    for N in LM:
        x, U = SolveEDP1(a, b, c, u_exact(a), u_exact(b), f, N)
        h = (b - a) / N
        H.append(h)
        erreur_max = np.max(np.abs(u_exact(x) - U))
        E.append(erreur_max)

    # Représentation graphique log-log
    plt.figure(figsize=(8, 6))
    plt.loglog(H, E, 'r<-', label='Erreur(h)', linewidth=2)
    plt.loglog(H, H, 'kd:', label='O(h)', linewidth=2)
    plt.loglog(H, np.array(H)**2, 'k*:', label='O(h²)', linewidth=2)
    ...
    plt.show()

# ---------------------------------------------------
# Lancement du test
# ---------------------------------------------------
test_convergence_edp()
```



- **la méthode numérique utilisée est un schéma aux différences finies centrées d'ordre 2 :**
  - **La courbe rouge (erreur) est parallèle à la droite « $O(h^2)$ ».**
- **Cela montre que :**
  - **Le schéma est consistant avec un ordre 2.**
  - **L'erreur décroît proportionnellement à h^2 lorsque le maillage s'affine.**

EDP stationnaire avec conditions aux limites mixtes

Problème modèle

- On considère le problème modèle suivant :

$$-u''(x) + c\,u(x) = f(x), \quad x \in [a, b]$$

- avec conditions aux limites mixtes :
  - **Dirichlet** à gauche : $u(a) = \alpha$
  - **Neumann** à droite : $u'(b) = \beta$
- où $a < b$, $c > 0$, $\alpha, \beta \in \mathbb{R}$, $f : [a, b] \to \mathbb{R}$ donnée.
- Ce problème est **bien posé**.

Discrétisation

- On discrétise $[a, b]$ en $N + 1$ points : $x_i = a + ih$, $i = 0, \dots, N$ avec pas $\Delta x = h = \frac{b-a}{N}$.
- On note $u_i \approx u(x_i)$.

Approximation de la dérivée seconde

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + O(h^2)$$

EDP stationnaire avec conditions aux limites mixtes

- En remplaçant u'' dans l'EDP :

$$-\frac{u_{i+1}-2u_i+u_{i-1}}{h^2} + cu_i = f(x_i) \text{ pour } i = 1, \dots, N-1 \ (*)$$

- On pose: $\mu = 2 + ch^2$

$$\Rightarrow -u_{i-1} + \mu u_i - u_{i+1} = h^2 f(x_i)$$

- **Condition de Dirichlet à gauche:** en $x_0 = a$: $u_0 = \alpha$
- **Condition de Neumann à droite: a*pproche à l'ordre 1***
  - x_{N+1} est hors domaine.
  - La dérivée en $x_N = b$ est approchée par la différence **arrière** :

$$u'(b) \approx \frac{u_N - u_{N-1}}{h} = \beta \quad \Rightarrow \quad u_N - u_{N-1} = h\beta$$

- Cela donne un schéma global **ordre 1**, car **l'approximation de Neumann est d'ordre 1** et l'erreur se propage à tout le domaine.

EDP stationnaire avec conditions aux limites mixtes

$$\Rightarrow \begin{cases} \textit{Pour } i = 0\textit{:} & u_0 = \alpha \\ \textit{Pour } i = 1\textit{:} & -u_2 + \mu u_1 - u_0 = h^2 f(x_1) \\ \textit{Pour } i = 2\textit{:} & -u_3 + \mu u_2 - u_1 = h^2 f(x_2) \\ \textit{Pour } i = 3\textit{:} & -u_4 + \mu u_3 - u_2 = h^2 f(x_3) \\ \textit{Pour } i = N-2\textit{:} & -u_{N-1} + \mu u_{N-2} - u_{N-3} = h^2 f(x_{N-2}) \\ \textit{Pour } i = N-1\textit{:} & -u_N + \mu u_{N-1} - u_{N-2} = h^2 f(x_{N-1}) \\ \textit{Pour } i = N\textit{:} & u_N - u_{N-1} = h\beta \end{cases}$$

Mise en système matriciel

- Si on assemble toutes les équations on obtient :

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 0 \\ -1 & \mu & -1 & \cdots & 0 & 0 \\ 0 & -1 & \mu & \ddots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \ddots & \mu & -1 \\ 0 & 0 & 0 & \cdots & -1 & 1 \end{bmatrix}}_{\breve{A}} \underbrace{\begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-1} \\ u_N \end{bmatrix}}_{\breve{U}} = \underbrace{\begin{bmatrix} \alpha \\ h^2 f(x_1) \\ h^2 f(x_2) \\ \vdots \\ h^2 f(x_{N-1}) \\ h\,\beta \end{bmatrix}}_{\breve{B}}$$

EDP stationnaire avec conditions aux limites mixtes

Listing: (Python) Example Extrait du script complet

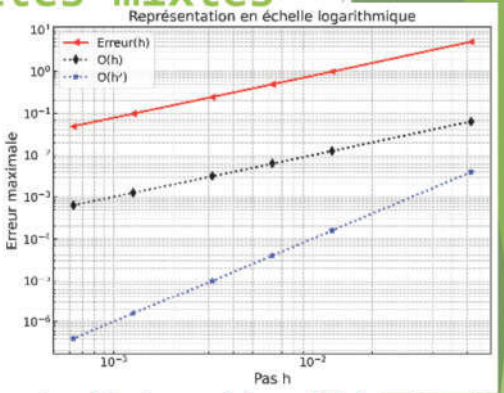
```
def test_convergence_edp():
    # Données du problème
    u_exact = lambda x: np.sin(x**2)  # Solution exacte
    du_exact = lambda x: 2 * x * np.cos(x ** 2)  # Derivative for
Neumann condition
    f = lambda x: 4*x**2*np.sin(x**2) - 2*np.cos(x**2) +
c*np.sin(x**2)
    a, b, c = 0, 2*np.pi, 1
    alpha = u_exact(a)
    beta = du_exact(b)

    # Différentes discrétisations
    LM = [100, 500, 1000, 2000, 5000, 10000]
    H = []
    E = []

    for N in LM:
        x, U = SolveEDP1(a, b, c, alpha, beta, f, N)
        h = (b - a) / N
        H.append(h)
        erreur_max = np.max(np.abs(u_exact(x) - U))
        E.append(erreur_max)

    # Représentation graphique log-log
    plt.figure(figsize=(8, 6))
    plt.loglog(H, E, 'r<-', label='Erreur(h)', linewidth=2)
    plt.loglog(H, H, 'kd:', label='O(h)', linewidth=2)
    plt.loglog(H, np.array(H)**2, 'b*:', label='O(h²)', linewidth=2)
    ...
    plt.show()

# ---------------------------------------------------
# Lancement du test
# ---------------------------------------------------
test_convergence_edp()
```



- **la méthode numérique utilisée est un schéma aux différences finies centrées d'ordre 1 :**
  - **La courbe rouge (erreur) est parallèle à la droite O(h).**
- **Cela montre que :**
  - **Le schéma est consistant avec un ordre 1.**
  - **L'erreur décroît proportionnellement à h^1 lorsque le maillage s'affine.**

EDP stationnaire avec conditions aux limites mixtes

Exercice

- Soit φ une fonction suffisamment régulière et $h > 0$.
- Q1.Montrer que :

$$\frac{d\varphi}{dx}(x) = \frac{-3\varphi(x) + 4\varphi(x+h) - \varphi(x+2h)}{2h} + \mathcal{O}(h^2)$$

$$\frac{d\varphi}{dx}(x) = \frac{3\varphi(x) - 4\varphi(x-h) + \varphi(x-2h)}{2h} + \mathcal{O}(h^2)$$

- *Q2. Déterminer une formule permettant de calculer une approximation à l'ordre 2 de* $\frac{d^2\varphi}{dx^2}(x)$ *en utilisant uniquement des valeurs de la fonction* φ *aux points* $x + ih$ *et* $x - ih$ *avec* $i \in \mathbb{N}$.

EDP stationnaire avec conditions aux limites mixtes

Correction

- R.1 On utilise le développement de Taylor :
  - En $x+h$: $\varphi(x+h) = \varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x) + \mathcal{O}(h^3)$
  - En $x+2h$: $\varphi(x+2h) = \varphi(x) + 2h\varphi'(x) + \frac{(2h)^2}{2}\varphi''(x) + \mathcal{O}(h^3)$
- On fait une combinaison linéaire :
- $4\varphi(x+h) - \varphi(x+2h) = 4\left[\varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x) + \mathcal{O}(h^3)\right] - [\varphi(x) + 2h\varphi'(x) +$

EDP stationnaire avec conditions aux limites mixtes

Correction

- R.1 On utilise le développement de Taylor :
  - En $x-h$: $\varphi(x-h)=\varphi(x)-h\varphi'(x)+\frac{h^2}{2}\varphi''(x)+\mathcal{O}(h^3)$
  - En $x-2h$: $\varphi(x-2h)=\varphi(x)-2h\varphi'(x)+\frac{(2h)^2}{2}\varphi''(x)+\mathcal{O}(h^3)$
- On fait une combinaison linéaire :
- $4\varphi(x-h)-\varphi(x-2h)=4\left[\varphi(x)-h\varphi'(x)+\frac{h^2}{2}\varphi''(x)+\mathcal{O}(h^3)\right]-[\varphi(x)-2h\varphi'(x)+$

EDP stationnaire avec conditions aux limites mixtes

Correction

- R.2 Approximation d'ordre 2 de $\frac{d^2\varphi}{dx^2}(x)$ en utilisant uniquement des points à droite de x, c'est-à-dire $\varphi(x+ih)$ avec $i \in \mathbb{N}$
  - $\varphi(x+h) = \varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x) + \mathcal{O}(h^3)$
  - $\varphi(x+2h) = \varphi(x) + 2h\varphi'(x) + 2h^2\varphi''(x) + \mathcal{O}(h^3)$

$$\Rightarrow \varphi(x+2h) - 2\varphi(x+h) = [\varphi(x) + 2h\varphi'(x) + 2h^2\varphi''(x)] - 2\left[\varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x)\right] + \mathcal{O}(h^3)$$

$$\Rightarrow \varphi(x+2h) - 2\varphi(x+h) = -\varphi(x) + h^2\varphi''(x) + \mathcal{O}(h^3)$$

$$\Rightarrow \boldsymbol{\varphi''(x) = \frac{\varphi(x) - 2\varphi(x+h) + \varphi(x+2h)}{h^2} + \mathcal{O}(h)}$$

- **Mais cette approximation est d'ordre 1.**
- Pour une approximation d'ordre 2, on utilise trois points en avant ou bien en arrière avec des coefficients corrects.

EDP stationnaire avec conditions aux limites mixtes

Correction

- R.2 Pour une approximation d'ordre 2, on utilise trois points en avant avec les coefficients corrects :
  - $\varphi(x+h) = \varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x) + \frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$
  - $\varphi(x+2h) = \varphi(x) + 2h\varphi'(x) + 2h^2\varphi''(x) + 8\frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$
  - $\varphi(x+3h) = \varphi(x) + 3h\varphi'(x) + 9\frac{h^2}{2}\varphi''(x) + 27\frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$

$$\Rightarrow \boldsymbol{-5\varphi(x+h) + 4\varphi(x+2h) - \varphi(x+3h)}$$

$$= -5\left[\varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(x) + \frac{h^3}{6}\varphi'''(x)\right] + 4\left[\varphi(x) + 2h\varphi'(x) + 2h^2\varphi''(x) + 8\frac{h^3}{6}\varphi'''(x)\right]$$

$$-\left[\varphi(x) + 3h\varphi'(x) + 9\frac{h^2}{2}\varphi''(x)) + 27\frac{h^3}{6}\varphi'''(x)\right] + \mathcal{O}(h^3)$$

$$\Rightarrow \boldsymbol{-5\varphi(x+h) + 4\varphi(x+2h) - \varphi(x+3h)} = -2\varphi(x) + h^2\varphi''(x) + \mathcal{O}(h^3)$$

- La formule d'ordre 2 à 3 points en avant est :

$$\boldsymbol{\varphi''(x) \approx \frac{2\varphi(x) - 5\varphi(x+h) + 4\varphi(x+2h) - \varphi(x+3h)}{h^2} + \mathcal{O}(h^2)}$$

EDP stationnaire avec conditions aux limites mixtes

Correction

- R.2 Approximation d'ordre 2 de $\frac{d^2\varphi}{dx^2}(x)$ en utilisant uniquement des points à gauche de x, c'est-à-dire $\varphi(x-ih)$
  - $\varphi(x-h) = \varphi(x) - h\varphi'(x) + \frac{h^2}{2}\varphi''(x) - \frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$
  - $\varphi(x-2h) = \varphi(x) - 2h\varphi'(x) + 2h^2\varphi''(x) - 8\frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$
  - $\varphi(x-3h) = \varphi(x) - 3h\varphi'(x) + \frac{9}{2}h^2\varphi''(x) - 27\frac{h^3}{6}\varphi'''(x) + \mathcal{O}(h^4)$

$$\Rightarrow \mathbf{-5\varphi(x+h) + 4\varphi(x+2h) - \varphi(x+3h)}$$

$$= -5\left[\varphi(x) - h\varphi'(x) + \frac{h^2}{2}\varphi''(x) - \frac{h^3}{6}\varphi'''(x)\right] + 4\left[\varphi(x) - 2h\varphi'(x) + 2h^2\varphi''(x) - 8\frac{h^3}{6}\varphi'''(x)\right]$$

$$-\left[\varphi(x) - 3h\varphi'(x) + 9\frac{h^2}{2}\varphi''(x)) - 27\frac{h^3}{6}\varphi'''(x)\right] + \mathcal{O}(h^3)$$

$$\Rightarrow \mathbf{-5\varphi(x-h) + 4\varphi(x-2h) - \varphi(x-3h)} = -2\varphi(x) + h^2\varphi''(x) + \mathcal{O}(h^3)$$

- La formule d'ordre 2 à 3 points en arrière est :

$$\Rightarrow \boldsymbol{\varphi''(x) = \frac{2\varphi(x) - 5\varphi(x-h) + 4\varphi(x-2h) - \varphi(x-3h)}{h^2}} + \mathcal{O}(h^2)$$

EDP stationnaire avec conditions aux limites mixtes

Amélioration : ordre 2 pour la condition de Neumann

$$-u_{i-1} + \mu u_i - u_{i+1} = h^2 f(x_i) \Rightarrow \textbf{ordre 2}$$

$$u'(b) \approx \frac{u_N - u_{N-1}}{h} = \beta \quad \Rightarrow \quad u_N - u_{N-1} = h\beta \Rightarrow \textbf{ordre 1}$$

$\Rightarrow$ schéma global d'ordre 1

- Pour retrouver un **schéma global d'ordre 2**, on améliore l'approximation de $u'(b) = \beta$ par une **formule à 3 points** (dérivée arrière d'ordre 2):
- **Formule obtenue** :

$$\varphi'(x) = \frac{3\varphi(x) - 4\varphi(x-h) + \varphi(x-2h)}{2h} + \mathcal{O}(h^2)$$

$$\Rightarrow u'(b) \approx \frac{3u_N - 4u_{N-1} + u_{N-2}}{2h} = \beta \Rightarrow \mathbf{3u_N - 4u_{N-1} + u_{N-2} = 2h\beta}$$

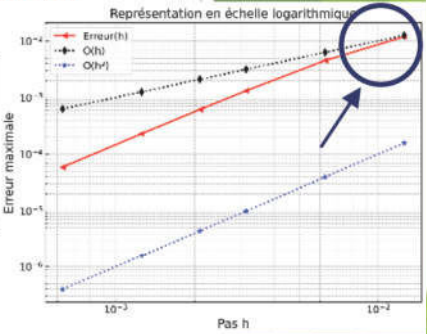
$$\Rightarrow \begin{cases} Pour\ i = 0 : u_0 = \alpha \\ Pour\ i = 1 : \ -u_2 + \mu u_1 - u_0 = h^2 f(x_1) \\ Pour i = 2 : \ -u_3 + \mu u_2 - u_1 = h^2 f(x_2) \\ Pour i = 3 : \ -u_4 + \mu u_3 - u_2 = h^2 f(x_3) \\ Pour i = N-2 : \ -u_{N-1} + \mu u_{N-2} - u_{N-3} = h^2 f(x_{N-2}) \\ Pour i = N-1 : \ -u_N + \mu u_{N-1} - u_{N-2} = h^2 f(x_{N-1}) \\ Pour\ i = N : \ 3u_N - 4u_{N-1} + u_{N-2} = 2h\beta \end{cases}$$

EDP stationnaire avec conditions aux limites mixtes

Amélioration : ordre 2 pour la condition de Neumann

- Si on assemble toutes les équations on obtient :

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 0 & 0 \\ -1 & \mu & -1 & \cdots & 0 & 0 & 0 \\ 0 & -1 & \mu & \ddots & 0 & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & 0 & \ddots & \vdots \\ 0 & 0 & 0 & \ddots & 0 & \mu & -1 \\ 0 & 0 & 0 & \cdots & 1 & -4 & 3 \end{bmatrix}}_{\tilde{A}} \underbrace{\begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-1} \\ u_N \end{bmatrix}}_{\tilde{U}} = \underbrace{\begin{bmatrix} \alpha \\ h^2 f(x_1) \\ h^2 f(x_2) \\ \vdots \\ h^2 f(x_{N-1}) \\ 2h\,\beta \end{bmatrix}}_{\tilde{B}}$$



- La méthode numérique utilisée est un schéma aux différences finies centrées **d'ordre 2** :
- La courbe rouge (erreur) est parallèle à la droite $\mathcal{O}(h^2)$.
- Le dernier point (grand h) ne suit pas la pente théorique car ce point est trop grossier pour que l'ordre d'approximation soit fiable:
  - Les termes d'erreur négligés dans le développement en série sont dominants pour h grand.

Problème modèle évolutif 1D : Équation de la chaleur

Problème modèle

- Trouver $u(t,x)$ défini sur $[0,T]\times[a,b]$ tel que :

$$\frac{\partial u}{\partial t}(t,x) - D\frac{\partial^2 u}{\partial x^2}(t,x) = f(t,x), \quad (t,x)\in[0,T]\times[a,b]$$

- avec les conditions initiales et aux limites :
  - Condition initiale :
    - $u(0,x) = u_0(x), \quad x\in[a,b]$
  - Condition de Neumann (flux) au bord gauche $x=a$:
    - $-D\frac{\partial u}{\partial x}(t,a) = \alpha(t), \quad t\in[0,T]$
  - Condition de Dirichlet au bord droit $x=b$:
    - $u(t,b) = \beta(t), \quad t\in[0,T]$

Problème modèle évolutif 1D : Équation de la chaleur

Discrétisation espace-temps

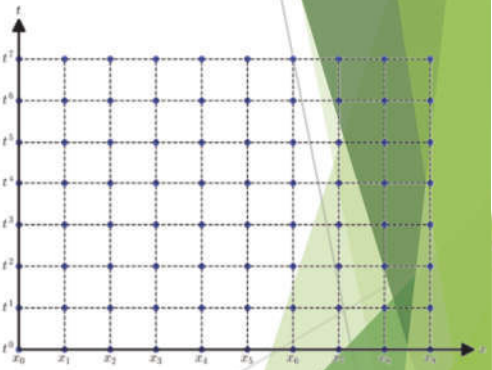
- Grille spatiale régulière avec $N_x + 1$ points :

$$x_i = a + i\Delta x, \quad i = 0, \dots, N_x, \quad \Delta x = \frac{b-a}{N_x}$$

- Grille temporelle régulière avec $N_t + 1$ points :

$$t_n = n\Delta t, \quad n = 0, \dots, N_t, \quad \Delta t = \frac{T}{N_t}$$

- Les valeurs approchées sont notées $u_i^n \approx u(t_n, x_i)$.



Représentation d'une grille espace-temps avec $N_t = 7$ et $N_x = 9$. Les noeuds de la grille sont les points bleus.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique explicite en temps

- Approximation des dérivées :
  - Dérivée en temps (avant) :

$$\frac{\partial u}{\partial t}(t_n, x_i) \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}$$

  - Dérivée seconde spatiale (centrée) :

$$\frac{\partial^2 u}{\partial x^2}(t_n, x_i) \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

L'equation EDP : $\frac{\partial u}{\partial t}(t,x) - D\frac{\partial^2 u}{\partial x^2}(t,x) = f(t,x)\ ,\quad (t,x) \in [0,T] \times [a,b]$

$$\Rightarrow \frac{\partial u}{\partial t}(t_n, x_i) - D\frac{\partial^2 u}{\partial x^2}(t_n, x_i) = f(t_n, x_i), \text{ pour } i = 0, \dots, N_x \text{ et } n = 0, \dots, N_t$$

$$\Rightarrow \frac{u_i^{n+1} - u_i^n}{\Delta t} - D\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} = f(t_n, x_i) = f_i^n$$

$$\Rightarrow u_i^{n+1} = u_i^n + E(u_{i+1}^n - 2u_i^n + u_{i-1}^n) + \Delta t\, f_i^n$$

$$\Rightarrow \boldsymbol{u_i^{n+1} = C\, u_i^n + E(u_{i+1}^n + u_{i-1}^n) + \Delta t\, f_i^n}, \quad \forall \boldsymbol{i} \in [\mathbf{1}, \boldsymbol{N_x - 1}] \text{ et } \forall \boldsymbol{n} \in [\mathbf{0}, \boldsymbol{N_t - 1}]$$

avec $E = D\frac{\Delta t}{\Delta x^2}$ et $C = 1 - 2E$

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique explicite en temps

Condition initiale

- $u(0,x) = u_0(x), \quad x \in [a,b] \Rightarrow \boldsymbol{u_i^0 = u_0(x_i)}, \ \forall \boldsymbol{i} \in [0, N_x]$

Conditions aux limites discrètes

- Au bord droit (Dirichlet) :

$$u(t,b) = \beta(t), \quad t \in (0,T] \Rightarrow \boldsymbol{u_{N_x}^{n+1} = \beta(t_{n+1}),\ \forall n \in [0, N_t - 1]}$$

- Au bord gauche (Neumann), approximation d'ordre 2 pour la dérivée en espace au temps t_{n+1} :

$$-D\frac{\partial u}{\partial x}(t,a) = \alpha(t), \quad t \in (0,T] \text{ or } \frac{\partial u}{\partial x}(t_{n+1}, x_0) \approx -\frac{-3u_0^{n+1} + 4u_1^{n+1} - u_2^{n+1}}{2\Delta x}$$

$$\Rightarrow -D\frac{-3u_0^{n+1} + 4u_1^{n+1} - u_2^{n+1}}{2\Delta x} = \alpha(t_{n+1})$$

$$\Rightarrow \boldsymbol{u_0^{n+1} = \frac{1}{3}\left(2\frac{\Delta x}{D}\alpha(t_{n+1}) + 4u_1^{n+1} - u_2^{n+1}\right)},\ \forall \boldsymbol{i} \in [0, N_x] \textbf{ et } \forall \boldsymbol{n} \in [0, N_t - 1]$$

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique explicite en temps

- En résumé, avec $E = D\frac{\Delta t}{\Delta x^2}$ et $C = 1 - 2E$:

$$u_i^{n+1} = C\,u_i^n + E(u_{i+1}^n + u_{i-1}^n) + \Delta t\, f_i^n, \quad \forall i \in [1, N_x - 1] \text{ et } \forall n \in [0, N_t - 1]$$

$$u_i^0 = u_0(x_i), \quad \forall i \in [0, N_x]$$

$$u_{N_x}^{n+1} = \beta(t_{n+1}), \ \forall n \in [0, N_t - 1]$$

$$u_0^{n+1} = \frac{1}{3}\left(2\frac{\Delta x}{D}\alpha(t_{n+1}) + 4u_1^{n+1} - u_2^{n+1}\right), \ \forall i \in [0, N_x] \text{ et } \forall n \in [0, N_t - 1]$$

- L'algorithme formel est donc le suivant :

1: $u_i^0 \leftarrow u_0(x_i)$, $\forall i \in \{0, \ldots, N_x\}$

2: **Pour** $n \leftarrow 0$ **à** $N_t - 1$ **faire**

3: $u_i^{n+1} \leftarrow C\,u_i^n + E(u_{i+1}^n + u_{i-1}^n) + \Delta t\, f_i^n$, $\forall i \in \{1, \ldots, N_x - 1\}$

4: $u_{N_x}^{n+1} \leftarrow \beta(t_{n+1})$

5: $u_0^{n+1} \leftarrow \frac{1}{3}\left(2\frac{\Delta x}{D}\alpha(t_{n+1}) + 4u_1^{n+1} - u_2^{n+1}\right)$

6: **Fin Pour**

Problème modèle évolutif 1D : Équation de la chaleur

Exercice:

- Q.1 Écrire une fonction algorithmique *Heat1Dex* permettant de retourner la discrétisation en temps, la discrétisation en espace, et l'ensemble des u_i^n, avec $i \in \{0,\dots,Nx\}$, $n \in \{1,\dots,Nt\}$, calculés par le schéma explicite en temps pour l'EDP de la chaleur.

$$u_i^{n+1} = C\,u_i^n + E(u_{i+1}^n + u_{i-1}^n) + \Delta t\, f_i^n, \quad \forall i \in [1, N_x - 1] \text{ et } \forall n \in [0, N_t - 1]$$

$$u_i^0 = u_0(x_i), \quad \forall i \in [0, N_x]$$

$$u_{N_x}^{n+1} = \beta(t_{n+1}), \ \forall n \in [0, N_t - 1]$$

$$u_0^{n+1} = \frac{1}{3}\left(2\frac{\Delta x}{D}\alpha(t_{n+1}) + 4u_1^{n+1} - u_2^{n+1}\right), \ \forall i \in [0, N_x] \text{ et } \forall n \in [0, N_t - 1]$$

- avec $E = D\frac{\Delta t}{\Delta x^2}$ et $C = 1 - 2E$
- Q.2 Écrire un programme utilisant cette fonction et permettant de calculer la solution numérique d'un problème dont on connaît la solution exacte

Problème modèle évolutif 1D : Équation de la chaleur

Correction:

- **Données d'entrée de la fonction *Heat1Dex* :**

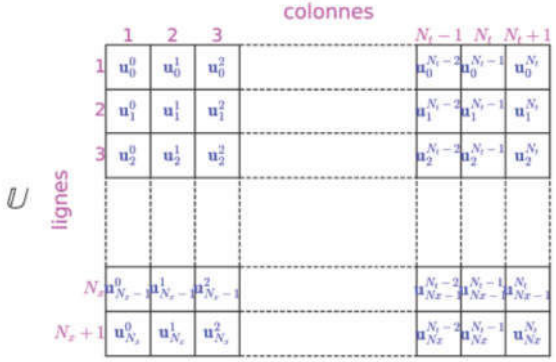
Nom de la donnée	Description
a, b	Deux réels avec $a < b$ définissant l'intervalle spatial $[a, b]$.
T	Temps final $T > 0$, définissant l'intervalle temporel $[0, T]$.
D	Coefficient de diffusion strictement positif $D > 0$.
$f(x, t)$	Terme source dépendant de l'espace et du temps, $f: [0, T] \times [a, b] \to \mathbb{R}$.
$u_0\,(x)$	Condition initiale sur l'espace, $u_0: [a, b] \to \mathbb{R}$.
$\alpha(t)$	Condition de Neumann en $x = a$, dérivée spatiale de u donnée en fonction du temps.
$\beta(t)$	Condition de Dirichlet en $x = b$, $u(b, t) = \beta(t)$, avec $\beta(0) = u_0(b)$.
N_x	Nombre de points de discrétisation de l'espace (hors conditions limites explicites si besoin), $Nx \in \mathbb{N}^*$.
N_t	Nombre de points de discrétisation du temps, $Nt \in \mathbb{N}^*$.

Problème modèle évolutif 1D : Équation de la chaleur

Correction:

- **Résultat de la fonction *Heat1Dex* :**

U : matrice (ou tableau 2D) de réels de taille $(Nx+1) \times (Nt+1)$ telle que : $U(i+1,\ n+1) = u_i^n, \quad \forall i \in \{0,\ldots,Nx\},\ \forall n \in \{0,\ldots,Nt\}$



Problème modèle évolutif 1D : Équation de la chaleur

Correction:

Algorithm Fonction *Heat1Dex* (version non vectorisée)

```
1:  Fonction [t, x, U] ← Heat1Dex(a, b, T, D, f, u0, α, β, Nx, Nt)
2:      t ← DisReg(0, T, Nt)                    → Discrétisation régulière en temps
3:      Δt ← T / Nt
4:      x ← DisReg(a, b, Nx)                    → Discrétisation régulière en espace
5:      Δx ← (b - a) / Nx
6:      E ← D * Δt / Δx²
7:      C ← 1 - 2 * E

8:      Pour i ← 1 à Nx + 1 faire               → Initialisation : condition initiale
9:          U(i, 1) ← u0(x(i))
10:     Fin Pour

11:     Pour n ← 1 à Nt faire                   → Boucle en temps
12:         Pour i ← 2 à Nx faire               → Schéma explicite pour i ∈ [1, Nx-1]
13:             U(i, n+1) ← C * U(i, n) + E * (U(i+1, n) + U(i-1, n)) + Δt * f(t(n), x(i))
14:         Fin Pour

15:         U(Nx+1, n+1) ← β(t(n+1))            → Condition de Dirichlet en x = b
16:         U(1, n+1) ← (Δx * α(t(n+1)) + 4 * U(2, n+1) - U(3, n+1)) / 3
                                                → Condition de Neumann d'ordre 2 en x = a
17:     Fin Pour

18: Fin Fonction
```

Problème modèle évolutif 1D : Équation de la chaleur

Correction:

Algorithm Fonction *Heat1Dex* (version vectorisée)

```
1:  Fonction [t, x, U] ← Heat1Dex(a, b, T, D, f, u0, α, β, Nx, Nt)

2:      t ← DisReg(0, T, Nt)                          → Vecteur temps
        Δt ← T / Nt
3:      x ← DisReg(a, b, Nx)                          → Vecteur espace
        Δx ← (b - a) / Nx
4:      E ← D * Δt / Δx²
        C ← 1 - 2 * E
5:      U[:, 1] ← u0(x)                               → Condition initiale (vectorisée)
6:      I ← [2 : Nx]                                  → Indices des points intérieurs
7:      Pour n ← 1 à Nt faire                         → Boucle en temps
8:          U[I, n+1] ← C * U[I, n]
                      + E * (U[I+1, n] + U[I-1, n])
                      + Δt * f(t[n], x[I])            → Schéma explicite (vectorisé)
9:          U[Nx+1, n+1] ← β(t[n+1])                  → Condition de Dirichlet
10:         U[1, n+1] ← (Δx * α(t[n+1]) + 4 * U[2, n+1] - U[3, n+1]) / 3
                                                      → Condition de Neumann d'ordre 2
11:     Fin Pour
12: Fin Fonction
```

Problème modèle évolutif 1D : Équation de la chaleur

Correction: **Listing: fonction pour la résolution de l'EDP de la chaleur par le schéma explicite**

```
import numpy as np

def Heat1DexLight(a, b, T, D, f, u0, alpha, beta, Nx, Nt):
    dt = T / Nt
    t = np.linspace(0, T, Nt + 1)

    dx = (b - a) / Nx
    x = np.linspace(a, b, Nx + 1)

    U = np.zeros((Nx + 1, Nt + 1))

    E = D * dt / dx**2
    C = 1 - 2 * E

    I = np.arange(1, Nx)  # i = 1 to Nx-1 (points intérieurs)

    # Condition initiale
    U[:, 0] = u0(x)

    # Boucle en temps
    for n in range(Nt):
        # Schéma explicite (points intérieurs)
        U[I, n + 1] = (
            C * U[I, n]
            + E * (U[I + 1, n] + U[I - 1, n])
            + dt * f(t[n], x[I])
        )

        # Condition de Dirichlet à droite
        U[Nx, n + 1] = beta(t[n + 1])

        # Condition de Neumann d'ordre 2 à gauche
        U[0, n + 1] = ((2 * dx / D) * alpha(t[n + 1]) + 4 * U[1, n + 1] - U[2, n + 1]) / 3

    return t, x, U
```

Problème modèle évolutif 1D : Équation de la chaleur

Application: problème avec solution exacte

- On fixe directement une solution particulière, par exemple :

$$u(t,x) = \cos(t)\cos(x)$$

- En injectant cette solution dans le système d'équations de la chaleur, on peut déterminer :
  - **Terme source** : $f(t,x) = \frac{\partial u}{\partial t}(t,x) - D\frac{\partial^2 u}{\partial x^2}(t,x) = D\cos(t)\cos(x) - \sin(t)\cos(x)$
  - **Condition initiale** : $u_0(x) = u(0,x) = \cos(x)$
  - **Condition de Neumann à gauche** : $\alpha(t) = -D\frac{\partial u}{\partial x}(t,a) = D\cos(t)\sin(a)$
  - **Condition de Dirichlet à droite** : $\beta(t) = u(t,b) = \cos(t)\cos(b)$
- On considère les valeurs suivantes pour effectuer la simulation :
  - $D = 1$ (diffusivité)
  - $a = 0$, $b = 2\pi$ (bornes de l'intervalle spatial)
  - $T = 10$ (temps final)
  - $N_x = 50$ (nombre de points en espace)
  - $N_t = 5000$ (nombre de points en temps)

Problème modèle évolutif 1D : Équation de la chaleur

Listing: (Python) Example Extrait du script complet

```
# Données du problème
a, b, T, Nx, Nt, D = 0, 2 * np.pi, 10, 50, 5000, 1

# Fonctions du problème
f = lambda t, x: D * np.cos(t) * np.cos(x) - np.sin(t) * np.cos(x)
u0 = lambda x: np.cos(x)
alpha = lambda t: D * np.cos(t) * np.sin(a)
beta = lambda t: np.cos(t) * np.cos(b)

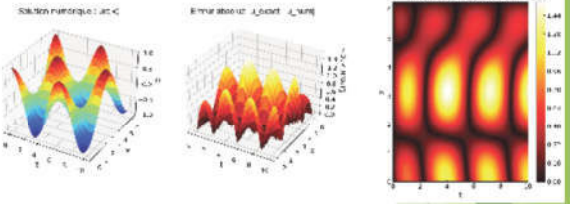
# Résolution numérique
t, x, U = Heat1DexLight(a, b, T, D, f, u0, alpha, beta, Nx, Nt)

# Calcul de la solution exacte sur la grille
T_grid, X_grid = np.meshgrid(t, x)
U_exact = np.cos(T_grid) * np.cos(X_grid)

# Calcul de l'erreur absolue
Erreur = np.abs(U_exact - U)

# Tracés des trois sous-figures
fig = plt.figure(figsize=(18, 12))

# 1. Solution numérique
ax1 = fig.add_subplot(131, projection='3d')
ax1.plot_surface(T_grid, X_grid, U, cmap='jet')
.....
# 2. Erreur absolue (3D)
ax2 = fig.add_subplot(132, projection='3d')
ax2.plot_surface(T_grid, X_grid, Erreur, cmap='hot')
.....
# 3. Erreur absolue (Contours) — version avec axes manuels
ax3 = fig.add_axes([0.72, 0.3, 0.25, 0.4])
contour = ax3.contourf(T_grid, X_grid, Erreur, levels=100,
cmap='hot')
cbar = fig.colorbar(contour, ax=ax3)
.....
```

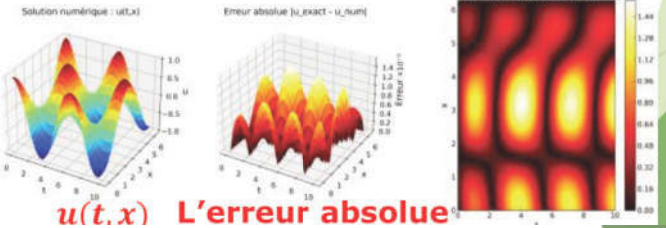


Problème modèle évolutif 1D : Équation de la chaleur

Application: problème avec solution exacte

- La surface de $u(t,x)$ est **lisse et régulière**, ce qui indique une bonne stabilité du schéma explicite utilisé.
- Le comportement périodique en x (lié à $\cos(x)$) et en t (lié à $\cos(t)$) est bien respecté.
- Aucune instabilité numérique visible.
- L'**erreur absolue=** $|u_{\text{exact}} - u_{\text{num}}|$ est très faible, de l'ordre de $10^{-4} - 10^{-3}$. Elle reste **localisée et uniforme**, sans pics violents.
- L'erreur vient très probablement :
  - Du **terme source** $f(t,x)$ mal intégré au pas de temps.
  - De la **discrétisation temporelle** (Euler explicite).

L'erreur absolue

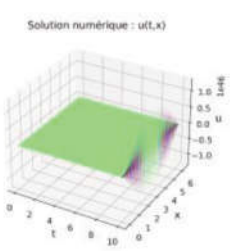


$u(t,x)$ **L'erreur absolue**

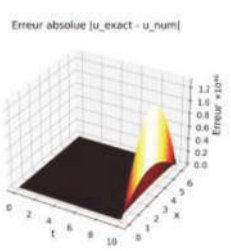
Problème modèle évolutif 1D : Équation de la chaleur

Application: N_x=100

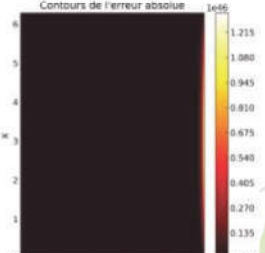
- On change uniquement N_x=50 a 100 ! A quoi peut-on s'attendre?
  - **Résolution de la solution → Plus fine, surface plus lisse**
  - **Erreur absolue → Réduction globale**
  - **Contours de l'erreur→Plus nets, erreurs plus localisées**
- Mais .....



$u(t,x)$



L'erreur absolue

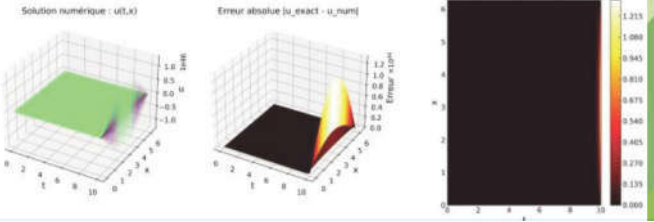


L'erreur absolue

Problème modèle évolutif 1D : Équation de la chaleur

Application: N_x=100

- A mesure que $t \to 10$:
  - On observe des pics de plus en plus grands dans $u(x,t)$
  - La surface devient chaotique ou bruitée
  - Les valeurs de u peuvent atteindre des valeurs absurdes (très grandes ou très négatives)
- Erreur absolue explose aussi avec le temps.
- L'erreur devient non localisée et forme des bandes non physiques.
- Les zones proches des bords (surtout avec Neumann) se dégénèrent rapidement.

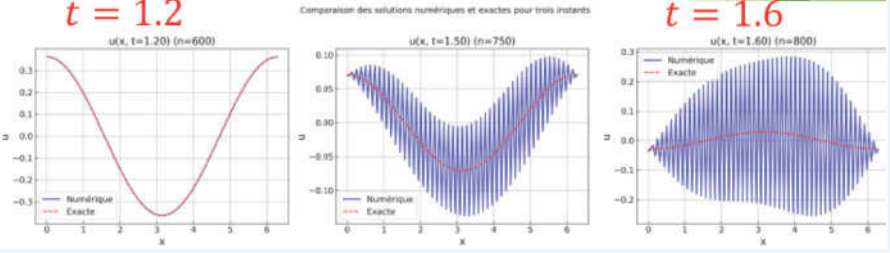


Problème modèle évolutif 1D : Équation de la chaleur

Application: N_x=100

- Pour mieux visualiser ce phénomène, on représente l'évolution de la solution numérique comparée à la solution exacte à trois instants clés.
  - À $t = 1.2$, la solution numérique coïncide avec la solution exacte.
  - À $t = 1.5$, de **légères oscillations parasites** apparaissent, signes d'une instabilité naissante.
  - À $t = 1.6$, des **pics non physiques** apparaissent : l'erreur explose localement. Cela confirme que le schéma explicite devient instable.
- Ces trois sous-graphes montrent clairement l'apparition progressive de l'**instabilité numérique**.

$t = 1.2$ $t = 1.5$ $t = 1.6$



Problème modèle évolutif 1D : Équation de la chaleur

Analyse de stabilité: méthode de Von Neumann

- Dans notre schéma : $E = \frac{D\,\Delta t}{\Delta x^2}$ et $C = 1 - 2E$
- Le schéma explicite discret s'écrit :

$$u_i^{n+1} = (1-2E)u_i^n + E(u_{i+1}^n + u_{i-1}^n) + \Delta t\, f(t_n, x_i)$$

  - Terme en f ne change pas la stabilité (source connue), donc on regarde uniquement le terme homogène.

Analyse de Von Neumann

- On suppose une solution harmonique : $\boldsymbol{u_i^n = G^n e^{jki\Delta x}}$
- où G est le facteur d'amplification.

Calcul des termes du schéma

- $u_i^{n+1} = G^{n+1} e^{jki\Delta x} = G^{n+1} e^{jki\Delta x}$
- $u_{i+1}^n = G^n e^{jk(i+1)\Delta x} = G^n e^{jki\Delta x} e^{jk\Delta x}$
- $u_{i-1}^n = G^n e^{jk(i-1)\Delta x} = G^n e^{jki\Delta x} e^{-jk\Delta x}$
- $u_i^n = G^n e^{jki\Delta x}$

Problème modèle évolutif 1D : Équation de la chaleur

Analyse de stabilité: méthode de Von Neumann

- En injectant dans le schéma :

$$u_i^{n+1} = (1-2E)u_i^n + Eu_{i+1}^n + Eu_{i-1}^n$$

$$\Rightarrow G^{n+1}e^{jki\Delta x} = (1-2E)G^n e^{jki\Delta x} + EG^n e^{jki\Delta x}(e^{jk\Delta x} + e^{-jk\Delta x})$$

$$\Rightarrow G^{n+1}e^{jki\Delta x} = G^n e^{jki\Delta x}\left[(1-2E) + E(e^{jk\Delta x} + e^{-jk\Delta x})\right]$$

$$\Rightarrow \frac{G^{n+1}}{G^n} = G = (1-2E) + E(e^{ik\Delta x} + e^{-ik\Delta x})$$

- Or, $e^{ik\Delta x} + e^{-ik\Delta x} = 2\cos(k\Delta x) \Rightarrow \boldsymbol{G = 1 - 2E + 2E\cos(k\Delta x)}$

Condition de stabilité

- Pour que le schéma soit stable, il faut : $|G| \leq 1$ pour tout k
- Le cas critique est pour $\cos(k\Delta x) = -1$: $\boldsymbol{G_{min} = 1 - 4E}$
- La condition $|G_{\min}| \leq 1$ donne :

$$-1 \leq 1 - 4E \leq 1 \Rightarrow \mathbf{0 \leq E \leq \frac{1}{2}}$$

Problème modèle évolutif 1D : Équation de la chaleur

analyse de stabilité: méthode de Von Neumann

Conclusion physique

- $E < \frac{1}{2}$ assure que toutes les ondes numériques ne sont pas amplifiées.
- En temps continu, ça se traduit par : $E = \frac{D\,\Delta t}{\Delta x^2} < \frac{1}{2} \Rightarrow \boldsymbol{\Delta t < \frac{\Delta x^2}{2D}}$
- Si $E \geq \frac{1}{2}$, les hautes fréquences explosent rapidement → instabilité.
- Le schéma **explicite** est **conditionnellement stable**, uniquement si :

$$E =< \frac{1}{2} \Rightarrow \boldsymbol{\Delta t < \frac{\Delta x^2}{2D}}$$

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

- Trouver

$$u(t_n, x_i) \in \mathbb{R}, \; n \in \{0, \dots, N_t\}, \; i \in \{0, \dots, N_x\},$$

- tels que

$$\frac{\partial u}{\partial t}(t_n, x_i) - D \frac{\partial^2 u}{\partial x^2}(t_n, x_i) = f(t_n, x_i), \quad n \in \{0, \dots, N_t\}, \quad i \in \{0, \dots, N_x\}$$

- avec conditions initiales et aux limites
  - $u(0, x_i) = u_0(x_i), \quad i \in \{0, \dots, N_x\}$
  - $-D \frac{\partial u}{\partial x}(t_n, x_0) = \alpha(t_n), \quad n \in \{0, \dots, N_t\}$
  - $u(t_n, x_{N_x}) = \beta(t_n), \quad n \in \{0, \dots, N_t\}$

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

- Approximation des dérivées :
  - Dérivée en temps (avant) :

$$\frac{\partial u}{\partial t}(t_n, x_i) \approx \frac{u(t_n, x_i) - u(t_{n-1}, x_i)}{\Delta t} = \frac{u_i^n - u_i^{n-1}}{\Delta t}$$

  - Dérivée seconde spatiale (centrée) :

$$\frac{\partial^2 u}{\partial x^2}(t_n, x_i) \approx \frac{u(t_n, x_{i+1}) - 2u(t_n, x_i) + u(t_n, x_{i-1})}{\Delta x^2} = \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}$$

- L'equation EDP : $\frac{\partial u}{\partial t}(t_n, x_i) - D\frac{\partial^2 u}{\partial x^2}(t_n, x_i) = f(t_n, x_i)$, pour $i = 0, \dots, N_x$ et $n = 0, \dots, N_t$

$$\Rightarrow \frac{u_i^n - u_i^{n-1}}{\Delta t} - D\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} = f_i^n$$

$$\Rightarrow \boldsymbol{C\, u_i^n - E(u_{i+1}^n + u_{i-1}^n) = u_i^{n-1} + \Delta t\, f_i^n}, \text{ avec } E = D\frac{\Delta t}{\Delta x^2} \text{ et } C = 1 + 2E$$

- Ce schéma est implicite en temps : il n'est pas possible de calculer explicitement u_i^n en fonction des u_i^{n-1} (au temps précédent).
- Toutefois, au temps t_n, les $N_x - 1$ équations sont linéaires en les $N_x + 1$ inconnues $(u_i^n)_{0 \le i \le N_x}$. Il manque donc 2 équations (linéaires) supplémentaires pour fermer le système, fournies par les conditions aux limites évaluées en t_n.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Condition initiale

$$u(0,x) = u_0(x), \quad x \in [a,b] \Rightarrow \boldsymbol{u_i^0 = u_0(x_i)}, \ \forall \boldsymbol{i} \in [0, N_x]$$

Conditions aux limites discrètes

- Au bord droit (Dirichlet) :

$$u(t,b) = \beta(t), \quad t \in (0,T] \Rightarrow \boldsymbol{u_{N_x}^n = \beta(t_n)}, \forall \boldsymbol{n} \in [0, N_t]$$

- Au bord gauche (Neumann), approximation d'ordre 2 pour la dérivée en espace au temps t_{n+1} :

$$-D\frac{\partial u}{\partial x}(t,a) = \alpha(t), \quad t \in (0,T]$$

$$\Rightarrow -D\frac{\partial u}{\partial x}(t_{n+1}, x_0) \approx -D\frac{-3u_0^n + 4u_1^n - u_2^{n1}}{2\Delta x} = \alpha(t_n)$$

$$\Rightarrow 3u_0^n - 4u_1^n + u_2^n = 2\frac{\Delta x}{D}\alpha(t_n), \ \forall \boldsymbol{i} \in [0, N_x] \text{ et } \forall \boldsymbol{n} \in [0, N_t]$$

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

- En résumé, pour $n \geq 1$, connaissant u_i^{n-1}, $i = 0, \ldots, N_x$, on calcule u_i^n en résolvant le système linéaire de $N_x + 1$ équations :

$$\begin{cases} \boldsymbol{3u_0^n - 4u_1^n + u_2^n = \dfrac{2\Delta x}{D}\alpha(t_n)} \\ \boldsymbol{Cu_i^n - E(u_{i+1}^n + u_{i-1}^n) = u_i^{n-1} + \Delta t f_i^n, \quad 1 \leq i \leq N_x - 1} \\ \boldsymbol{u_{N_x}^n = \beta(t_n)} \end{cases}$$

- Avec $E = D\frac{\Delta t}{\Delta x^2}$, $\quad C = 1 + 2E.$
- La forme matricielle s'écrit: $\boldsymbol{AU^n = b^n}$, avec $A \in \mathbb{R}^{(N_x+1)\times(N_x+1)}$ et $b^n \in \mathbb{R}^{N_x+1}$

$$U^n = \begin{pmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{N_x-1}^n \\ u_{N_x}^n \end{pmatrix}, \quad A = \begin{pmatrix} 3 & -4 & 1 & 0 & \cdots & 0 \\ -E & C & -E & 0 & \cdots & 0 \\ 0 & -E & C & -E & \cdots & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & \cdots & 0 & -E & C & -E \\ 0 & \cdots & 0 & 0 & 0 & 1 \end{pmatrix}, \text{ et } b^n = \begin{pmatrix} \frac{2\Delta x}{D}\alpha(t_n) \\ u_1^{n-1} + \Delta t f_1^n \\ \vdots \\ u_{N_x-1}^{n-1} + \Delta t f_{N_x-1}^n \\ \beta(t_n) \end{pmatrix}$$

- A est tridiagonale qui ne depend pas de temps n.
- ,

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Algorithmique

- Comme pour le schéma explicite, l'initialisation est donnée par la condition initiale :

$$U_i^0 = u_0(x_i), \quad i = 0, \dots, N_x.$$

- L'algorithme formel est donc :

1: Initialiser $U_i^0 \leftarrow u_0(x_i), i = 0, \dots, N_x$.

2: Pour $n \leftarrow 1$ à N_t faire

3: Construire la matrice A.

4: Calcul de b^n.

5: Résoudre le système linéaire $AU^n = b^n$.

6: Fin Pour.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Algorithmique

- Comme pour le schéma explicite, l'initialisation est donnée par la condition initiale :

$$U_i^0 = u_0(x_i), \quad i = 0, \dots, N_x.$$

- L'algorithme formel est donc :

1: Initialiser $U_i^0 \leftarrow u_0(x_i), i = 0, \dots, N_x$.

2: Pour $n \leftarrow 1$ à N_t faire

3: Construire la matrice A.

4: Calcul de b^n.

5: Résoudre le système linéaire $AU^n = b^n$.

6: Fin Pour.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- Q.1 Écrire la fonction ***AssembleMatGen1D*** retournant la matrice $M \in \mathcal{M}_d(\mathbb{R})$ définie par :

$$M = \begin{pmatrix} a_1 & a_2 & a_3 & 0 & \cdots & 0 \\ \beta & \alpha & \beta & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \beta & \alpha & \beta \\ 0 & \cdots & 0 & b_3 & b_2 & b_1 \end{pmatrix}$$ où $\alpha, \beta, a_1, a_2, a_3, b_1, b_2, b_3$ sont des réels donnés.

- Q.2 Écrire la fonction ***SndMbrGen1d*** retournant le vecteur $B \in \mathbb{R}^d$ défini par :

$$B = \begin{pmatrix} \alpha \\ c_1 \\ \vdots \\ c_{d-2} \\ \beta \end{pmatrix}$$ où $\alpha, \beta, c_1, \ldots, c_{d-2}$ sont des réels donnés.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- **R.1** On a par exemple l'algorithme suivant

Algorithme Fonction AssembleMatGen1D

Données :
d : dimension de la matrice,
$\alpha, \beta \in \mathbb{R}$,
$a, b \in \mathbb{R}^3$ tels que $a(i)=a_i$, $b(i)=b_i$.
Résultat :
$M \in \mathbb{R}^{(d \times d)}$

```
1: Fonction M ← AssembleMatGen1D(d, a, b, α, β)
2: M ← matrice nulle de taille d × d
3: M(1, 1) ← a(1), M(1, 2) ← a(2), M(1, 3) ← a(3)
4: M(d, d) ← b(1), M(d, d−1) ← b(2), M(d, d−2) ← b(3)
5: Pour i = 2 à d−1 faire
6:     M(i, i) ← α
7:     M(i, i−1) ← β
8:     M(i, i+1) ← β
9: Fin Pour
10: Fin Fonction
```

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- **R.1** On a par exemple l'algorithme suivant

Algorithme Fonction SndMbrGen1d

Données :
    d : dimension du vecteur,
    $\alpha, \beta \in R$,
    $c \in R^{(d-2)}$ tel que $c(i)=c_i$
Résultat :
    $B \in R^d$
1: Fonction B ← SndMbrGen1d(d, c, α, β)
2: $B(1) \leftarrow \alpha$
3: Pour i = 2 à d−1 faire
4: $B(i) \leftarrow c(i-1)$
5: Fin Pour
6: $B(d) \leftarrow \beta$
7: Fin Fonction

Problème modèle évolutif 1D : Équation de la chaleur

Correction:

Listing: fonction AssembleMatGen1D

```
def AssembleMatGen1D(d, a, b, alpha, beta):
    A = np.zeros((d, d))
    A[0, 0:3] = a
    A[-1, -3:] = b[::-1]
    for i in range(1, d - 1):
        A[i, i - 1] = beta
        A[i, i]     = alpha
        A[i, i + 1] = beta
    return A
```

Listing: fonction SndMbrGen1d

```
def SndMbrGen1d(d, c, alpha, beta):
    B = np.zeros(d)
    B[0] = alpha
    B[1:-1] = c
    B[-1] = beta
    return B
```

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- Q.1 Écrire une fonction algorithmique Heat1Dim permettant de retourner la discrétisation en temps, la discrétisation en espace et l'ensemble des u_i^n, pour $i \in \{0, \dots, N_x\}$ et $n \in \{1, \dots, N_t\}$, calculés par le schéma implicite.
- La forme matricielle s'écrit: $\boldsymbol{AU^n = b^n}$, avec $A \in \mathbb{R}^{(N_x+1)\times(N_x+1)}$ et $b^n \in \mathbb{R}^{N_x+1}$

$$U^n = \begin{pmatrix} u_0^n \\ u_1^n \\ \vdots \\ u_{N_x-1}^n \\ u_{N_x}^n \end{pmatrix}, \quad A = \begin{pmatrix} 3 & -4 & 1 & 0 & \cdots & 0 \\ -E & C & -E & 0 & \cdots & 0 \\ 0 & -E & C & -E & \cdots & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & \cdots & 0 & -E & C & -E \\ 0 & \cdots & 0 & 0 & 0 & 1 \end{pmatrix}, \text{ et } b^n = \begin{pmatrix} \frac{2\Delta x}{D}\alpha(t_n) \\ u_1^{n-1} + \Delta t f_1^n \\ \vdots \\ u_{N_x-1}^{n-1} + \Delta t f_{N_x-1}^n \\ \beta(t_n) \end{pmatrix}$$

- Q.2 Écrire un programme utilisant cette fonction et permettant de calculer la solution numérique d'un problème dont on connaît la solution exacte.

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- **R.1** On a par exemple l'algorithme suivant

Algorithme: Fonction Heat1Dim (non vectorisée)

```
1: Fonction [t, x, U] ← Heat1Dim(a, b, T, D, f, u0, α, β, Nx, Nt)
2: t ← DisReg(0, T, Nt) → Vecteur temps
3: Δt ← T / Nt
4: x ← DisReg(a, b, Nx) → Vecteur espace
5: Δx ← (b - a) / Nx
6: E ← D * Δt / Δx², C ← 1 + 2E
7: Pour i de 1 à Nx+1 faire
8:     U(i, 1) ← u0(x(i))    # Condition initiale
9: Fin Pour
10: A ← AssembleMatGen1D([3, -4, 1], [1, 0, 0], C, -E)
11: Pour n de 1 à Nt faire        # Boucle en temps
12:     Pour i de 1 à Nx-2 faire
13:         v(i) ← U(i+1, n) + Δt * f(t(n), x(i))
14:     Fin Pour
15:     L ← (2Δx / D) * α(t(n))
16:     B ← SndMbrGen1D(v, L, β(t(n)))
17:     U(:, n+1) ← Solve(A, B)
18: Fin Pour
19: Fin Fonction
```

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- **R.1** On a par exemple l'algorithme suivant

Algorithme: Fonction Heat1Dim (vectorisée)

```
1: Fonction [t, x, U] ← Heat1Dim(a, b, T, D, f, u0, α, β, Nx, Nt)
2: t ← DisReg(0, T, Nt) → Vecteur temps
3: Δt ← T / Nt
4: x ← DisReg(a, b, Nx) → Vecteur espace
5: Δx ← (b - a) / Nx
6: E ← D * Δt / Δx², C ← 1 + 2E
7: U(:, 1) ← u0(x)        # Condition initiale (vectorisée)
8: A ← AssembleMatGen1D([3, -4, 1], [1, 0, 0], C, -E)
9: Pour n de 1 à Nt faire   # Boucle en temps
10:     v ← U(2:Nx, n) + Δt * f(t(n), x(2:Nx))
11:     L ← (2Δx / D) * α(t(n))
12:     B ← SndMbrGen1D(v, L, β(t(n)))
13:     U(:, n+1) ← Solve(A, B)
14: Fin Pour
15: Fin Fonction
```

Problème modèle évolutif 1D : Équation de la chaleur

Correction:

Listing: fonction Heat1Dim

```
def Heat1Dim(a, b, T, D, f, u0, alpha_fun, beta_fun, Nx, Nt):
    x = np.linspace(a, b, Nx + 1)       # Steps 4–5
    t = np.linspace(0, T, Nt + 1)       # Steps 2–3
    dx = (b - a) / Nx
    dt = T / Nt
    E = D * dt / dx**2                  # Step 6
    C = 1 + 2 * E

    U = np.zeros((Nx + 1, Nt + 1))      # Init
    U[:, 0] = u0(x)                     # Step 7

    A = AssembleMatGen1D(Nx + 1, [3, -4, 1], [1, 0, 0], C, -E)
# Step 8

    for n in range(Nt):                 # Step 9
        tn = t[n+1]
        v = U[1:Nx, n] + dt * f(tn, x[1:Nx])        # Step 10
        L = (2 * dx / D) * alpha_fun(tn)            # Step 11
        rhs = SndMbrGen1d(Nx + 1, v, L, beta_fun(tn)) # Step
12
        U[:, n+1] = np.linalg.solve(A, rhs)         # Step 13
    return t, x, U
```

Problème modèle évolutif 1D : Équation de la chaleur

Schéma numérique implicite en temps

Exercice

- **R.2** On a par exemple l'algorithme suivant

Algorithme:

```
1:  a ← 0 ; b ← 1
2:  D ← 0.1
3:  Nx ← 50 ; Nt ← 1000      ▷ Nb de points espace/temps
4:  Δx ← (b − a) / Nx
5:  T ← 10          ▷ temps final
6:  Δt ← T / Nt

7: u0(x)   ← sin(pi * x)      ▷ condition initiale
8: f(t,x)  ← 0                ▷ car u_ex satisfait u_t = D u_xx
9: alpha(t) ← 0  et  beta(t) ← 0     ▷ Dirichlet homogène

10:  [t, x, U] ← Heat1Dim(a, b, T, D, f, u0, alpha, beta, Nx, Nt)
```

Algorithme: suite optionnel

```
11:  # Évaluer la sol exacte au temps final t_end = T
12:  Pour i de 0 à Nx faire
13:      Uex(i) ← sin(pi * x(i)) * exp( - n² * D * T )
14:  Fin Pour

15:  # Calcul de l'erreur
16:  Pour i de 0 à Nx faire
17:      erreur(i) ← | U(i, Nt) − Uex(i) |
18:  Fin Pour
19:  err_max ← max_i erreur(i)

20:  Afficher "Erreur maximale =", err_max

21:  # Tracé
22:  Plot(x, U(:, Nt))  ▷ sol numérique (trait continu)
23:  Plot(x, Uex)       ▷ sol exacte    (pointillés)
```

Problème modèle évolutif 1D : Équation de la chaleur

Application: problème avec solution exacte

- Utilisons la même solution particulière: $\boldsymbol{u(t,x) := \cos(t)\cos(x)}$
  - **Terme source** : $f(t,x) := D\cos(t)\cos(x) - \sin(t)\cos(x)$
  - **Condition initiale** : $u_0(x) := \cos(x)$
  - **Condition de Neumann à gauche** : $\alpha(t) := D\cos(t)\sin(a)$
  - **Condition de Dirichlet à droite** : $\beta(t) := \cos(t)\cos(b)$
- On considère les mêmes valeurs pour effectuer la simulation :
  - $D = 1$ (diffusivité)
  - $a = 0$, $b = 2\pi$ (bornes de l'intervalle spatial)
  - $T = 10$ (temps final)
  - $\boldsymbol{N_x = 100}$ (nombre de points en espace)
  - $N_t = 5000$ (nombre de points en temps)

Problème modèle évolutif 1D : Équation de la chaleur

Listing: (Python) Example Extrait du script complet

```
# Données du problème
a, b, T, Nx, Nt, D = 0, 2 * np.pi, 10, 100, 5000, 1

# Fonctions exactes et conditions aux bords
f = lambda t, x: D * np.cos(t) * np.cos(x) - np.sin(t) * np.cos(x)
u0 = lambda x: np.cos(x)
alpha = lambda t: D * np.cos(t) * np.sin(a)
beta = lambda t: np.cos(t) * np.cos(b)

# Résolution numérique (schéma implicite)
t, x, U = Heat1Dim(a, b, T, D, f, u0, alpha, beta, Nx, Nt)

# Solution exacte
T_grid, X_grid = np.meshgrid(t, x)
U_exact = np.cos(T_grid) * np.cos(X_grid)

# Erreur absolue
Erreur = np.abs(U_exact - U)

# ---------- Affichages ----------
fig = plt.figure(figsize=(18, 12))

# 1. Solution numérique
ax1 = fig.add_subplot(131, projection='3d')
ax1.plot_surface(T_grid, X_grid, U, cmap='jet')
.....
# 2. Erreur absolue
ax2 = fig.add_subplot(132, projection='3d')
ax2.plot_surface(T_grid, X_grid, Erreur, cmap='hot')
.....
# 3. Contour de l'erreur
ax3 = fig.add_axes([0.72, 0.3, 0.25, 0.4])
contour = ax3.contourf(T_grid, X_grid, Erreur, levels=100,
cmap='hot')
.....
plt.show()
```

Schéma explicite d'Euler ($N_x = 100$)

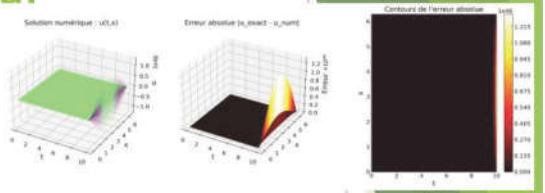
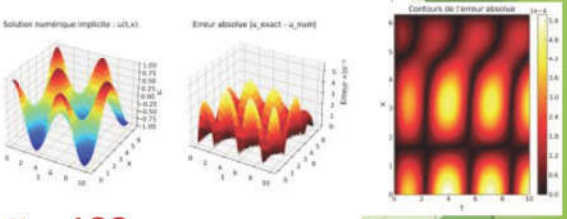


Schéma implicite d'Euler (N_x=100)



N_x=100

- Le schéma implicite est inconditionnellement stable.

Problème modèle évolutif 1D : Équation de la chaleur

Analyse de stabilité: méthode de Von Neumann

- Dans notre schéma : $E = \frac{D\,\Delta t}{\Delta x^2}$ et $C = 1 + 2E$
- Le schéma explicite discret s'écrit :

$$C u_i^n - E(u_{i+1}^n + u_{i-1}^n) = u_i^{n-1} + \Delta t f_i^n$$

- Terme en f ne change pas la stabilité (source connue), donc on regarde uniquement le terme homogène.

Analyse de Von Neumann

- On suppose une solution harmonique : $\boldsymbol{u_i^n = G^n e^{jkx_i} = G^n e^{jki\Delta x}}$
- où G est le facteur d'amplification.

Calcul des termes du schéma

- $u_i^{n-1} = G^{n+1} e^{jki\Delta x} = G^{n-1} e^{jki\Delta x}$
- $u_{i+1}^n = G^n e^{jk(i+1)\Delta x} = G^n e^{jki\Delta x} e^{jk\Delta x}$
- $u_{i-1}^n = G^n e^{jk(i-1)\Delta x} = G^n e^{jki\Delta x} e^{-jk\Delta x}$
- $u_i^n = G^n e^{jki\Delta x}$

Problème modèle évolutif 1D : Équation de la chaleur

Analyse de stabilité: méthode de Von Neumann

- En injectant dans le schéma : $Cu_i^n - E(u_{i+1}^n + u_{i-1}^n) = u_i^{n-1}$

$$\Rightarrow G^n e^{jki\Delta x} - E\left(G^n e^{jki\Delta x} e^{jk\Delta x} + G^n e^{jki\Delta x} e^{-jk\Delta x}\right) = G^{n-1} e^{jki\Delta x}$$

$$\Rightarrow G^n e^{jki\Delta x}\left[\mathrm{C} - \mathrm{E}(e^{jk\Delta x} + e^{-jk\Delta x})\right] = G^{n-1} e^{jki\Delta x}$$

$$\Rightarrow \frac{G^{n-1}}{G^n} = \frac{1}{G} = \mathrm{C} - \mathrm{E}(e^{jk\Delta x} + e^{-jk\Delta x})$$

- Or $e^{ik\Delta x} + e^{-ik\Delta x} = 2\cos(k\Delta x) \Rightarrow \boldsymbol{G = \frac{1}{\mathrm{C} - 2E\cos(k\Delta x)} = \frac{1}{1 + 2E(1 - \cos(k\Delta x))}}$
- $\cos(k\Delta x) \in [-1,1] \Rightarrow 1 - \cos(k\Delta x) \in [0,2] \Rightarrow 2E\left(1 - \cos(k\Delta x)\right) \geq 0$

$$\Rightarrow \mathbf{0 < |G| \leq 1}$$

  - Le facteur d'amplification $|G| \leq 1$ pour tout k.
  - Donc, le schéma implicite est inconditionnellement stable au sens de Von Neumann.